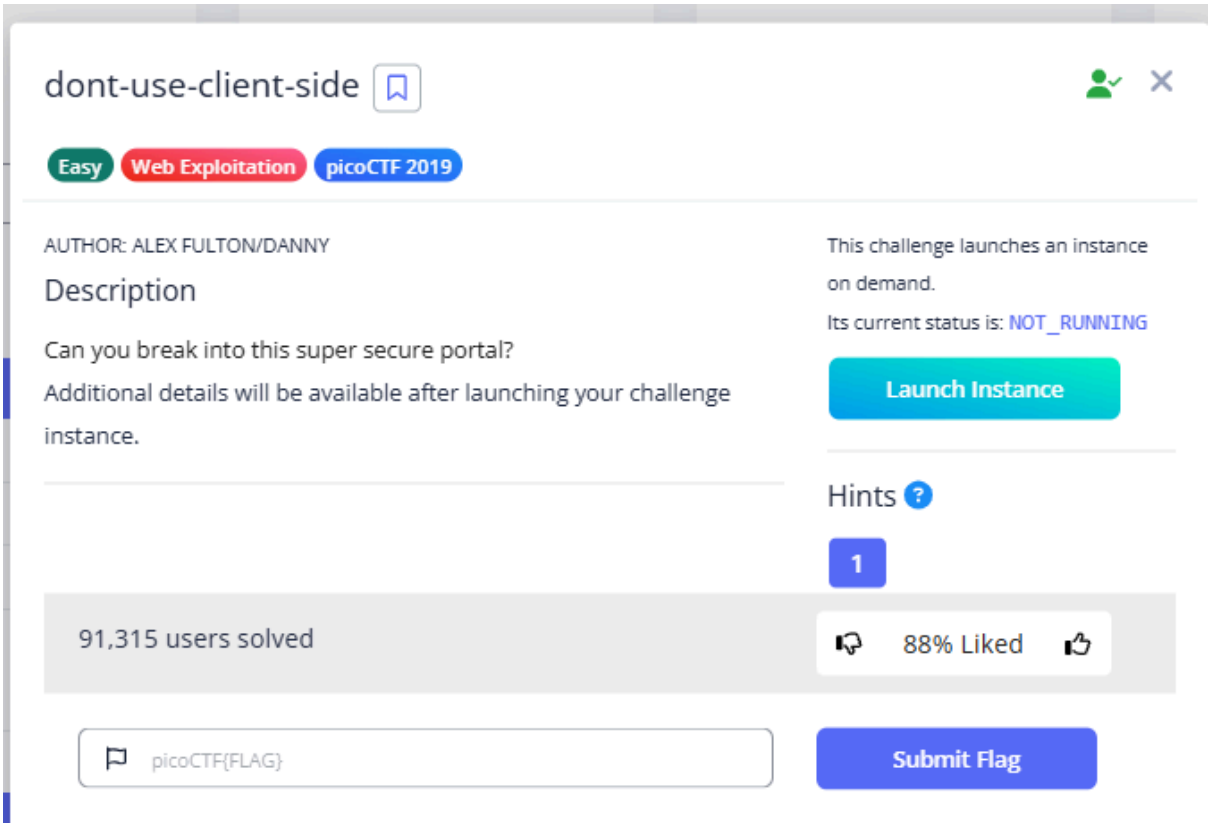
JASMINE C. OMANDAM

picoCTF - picoGym Challenges




```
<html>
<head>
<title>Secure Login Portal</title>
</head>
<body bgcolor=blue>
<!-- standard MD5 implementation -->
<script type="text/javascript" src="md5.js"></script>

<script type="text/javascript">
  function verify() {
    checkpass = document.getElementById("pass").value;
    split = 4;
    if (checkpass.substring(0, split) == 'pico') {
      if (checkpass.substring(split*6, split*7) == 'eb02') {
        if (checkpass.substring(split, split*2) == 'CTF{') {
         if (checkpass.substring(split*4, split*5) == 'ts_p') {
          if (checkpass.substring(split*3, split*4) == 'lien') {
            if (checkpass.substring(split*5, split*6) == 'lz_2') {
              if (checkpass.substring(split*2, split*3) == 'no_c') {
                if (checkpass.substring(split*7, split*8) == 'b45}') {
                  alert("Password Verified")
                }
              }
            }

          }
         }
        }
      }
    }
    else {
      alert("Incorrect password");
    }

  }
</script>
<div style="position:relative; padding:5px;top:50px; left:38%; width:350px; height:140px; background-color:yellow">
<div style="text-align:center">
```

Write - Up: dont-use-client-side

The challenge *“dont-use-client-side”* presented a login portal that claimed to be secure. At first glance, it looked like a normal password box with a “verify” button. However, the hint in the challenge title suggested that the validation logic was happening on the client side, meaning the password check was exposed in the JavaScript code.

Step 1: Inspecting the Page

I opened the portal and saw a simple yellow login box asking for credentials. Since no username or password was provided, the next step was to inspect the source code.

Step 2: Reading the JavaScript

Inside the HTML, I found a script block containing the `verify()` function. This function checked the input password by comparing substrings of 4 characters against hardcoded values. The code looked like this:

javascript
```
if (checkpass.substring(0, 4) == 'pico')
if (checkpass.substring(4, 8) == 'CTF{')
if (checkpass.substring(8, 12) == 'no_c')
if (checkpass.substring(12, 16) == 'lien')
if (checkpass.substring(16, 20) == 'ts_p')
if (checkpass.substring(20, 24) == 'lz_2')
if (checkpass.substring(24, 28) == 'eb02')
if (checkpass.substring(28, 32) == 'b45}')
```

Step 3: Reconstructing the Password

By combining these chunks in order, the full password was revealed:

picoCTF{no_clients_plz_2eb02b45}

Notice that the phrase “no_client_side_plz” was split across multiple substrings (`no_c`, `lien`, `ts_p`, `lz_2`). When joined together, it formed the readable message.

Step 4: Entering the Flag

Typing this exact string into the login box and clicking verify triggered the “Password Verified” alert, confirming that the flag was correct.

This challenge demonstrated why client-side validation is insecure. By placing the password check in JavaScript, the developers exposed the entire logic to anyone who inspected the source code. In real-world applications, authentication must always be handled server-side to prevent attackers from simply reading the code and reconstructing the password. The lesson here is clear: never trust the client with sensitive validation.

Final Flag: picoCTF{no_clients_plz_2eb02b45}